Module 8- Web Technologies in Java

HTML Tags: Anchor, Form, Table, Image, List Tags, Paragraph, Break, Label

Que: 1

Introduction to HTML and Its Structure

Ans:

HTML (Hyper Text Markup Language) is the standard language used to create web pages. It defines the structure of a webpage using different tags. HTML tells the browser how to display text, images, links, forms, tables, and other content.

- **Basic Structure of an HTML Document**

```
<!DOCTYPE html>
<html>
<head>
  <title>My Web Page</title>
</head>
<body>
  Content goes here
</body>
</html>
```

- **Explanation:**

  <!DOCTYPE html> → Defines the document type.

  <html> → Root element of HTML.

  <head> → Contains page information (title, meta data).

  <body> → Contains visible content of the webpage.

Que: 2

Explanation of Key HTML Tags.

Ans:

1. **<a> – Anchor Tag**

- Used to create hyperlinks that connect one page to another.

  Ex: Visit Website

2. **<form> – Form Tag**

- Used to collect user input like name, email, password, etc.

  Ex: <form>

  <input type="text" name="Enter Name">

  </form>

3. **<table> – Table Tag**

- Used to display data in rows and columns.

  Ex: <table border="1">

  <tr>

  <th>Name</th>

  <th>Age</th>

  </tr>

  <tr>

  <td>Rahul</td>

  <td>22</td>

  </tr>

  </table>

4. **<img> – Image Tag**

- Used to add images to a webpage.

  Ex: <img src="image.jpg" alt="Sample Image" width="200">

5. **List Tags – , <ol>, **

- Used to create lists.
  - **Unordered List (<ul>)**

    Ex:

    HTML

    CSS

  - **Ordered List (<ol>)**

    Ex:

    HTML

    CSS

6. ** – Paragraph Tag**

- Used to display a paragraph of text.

  Ex: This is a paragraph.

7. **
 – Line Break**

- Used to move content to the next line.

  Ex: Hello
World

8. **<label> – Label Tag**

- Used to give a name to form input fields.

  Ex: <label>Name:</label>

  <input type="text">

CSS: Inline CSS, Internal CSS, External CSS

Que: 1

Overview of CSS and Its Importance in Web Design

Ans:

CSS (Cascading Style Sheets) is used to control the appearance and layout of web pages. While HTML defines the structure of a webpage, CSS is responsible for colors, fonts, spacing, borders, alignment, and responsiveness.

- **Importance of CSS in Web Design**
- Improves the visual appearance of websites
- Keeps design separate from content
- Makes web pages consistent and professional
- Helps in responsive design for mobile and desktop
- Reduces code repetition and saves time

Que: 2

Types of CSS:

- **o Inline CSS: Directly in HTML elements.**
- **o Internal CSS: Inside a <style> tag in the head section.**
- **o External CSS: Linked to an external file.**

Ans:

- **There are three main types of CSS used in web design:**

1) **Inline CSS**
   - Inline CSS is written directly inside an HTML element using the style attribute.
   - It affects only one element.

- **Example:**

```
<p style="color: blue; font-size: 18px;">
  This is Inline CSS
</p>
```

- **Advantages:**
  - Quick styling for small changes
- **Disadvantages:**
  - Not reusable
  - Makes HTML code messy

2) **Internal CSS**
   - Internal CSS is written inside a <style> tag within the <head> section of the HTML file.
   - It applies styles to one entire page.

- **Example:**

```
<!DOCTYPE html>
<html>
<head>
  <style>
```

```
        p {
            color: green;
            font-size: 16px;
        }
    </style>
</head>
<body>
    <p>This is Internal CSS</p>
</body>
</html>
```

- **Advantages:**
  - Useful for single-page styling
  - Cleaner than inline CSS
- **Disadvantages:**
  - Not suitable for large websites

3) External CSS

  - External CSS is written in a separate .css file and linked to the HTML file using the <link> tag.
  - It is the most recommended method.

- **HTML File:**

```
<link rel="stylesheet" href="style.css">
```

- **CSS File (style.css):**

```
p {
    color: red;
    font-size: 18px;
}
```

- **Advantages:**
  - Reusable across multiple pages
  - Easy maintenance
  - Clean and professional code

CSS: Margin and Padding

Que: 1

Definition and difference between margin and padding.

Ans:

❖ **Definition of Margin and Padding**

- **Margin**
  - Margin is the space outside an HTML element's border.
  - It is used to create distance between elements.
- **Padding**
  - Padding is the space inside an HTML element's border.
  - It creates space between the content and the border.

➢ **Difference Between Margin and Padding**

Feature	**Margin**	**Padding**
Space location	Outside the border	Inside the border
Purpose	Space between elements	Space around content
Background color	Not affected	Affected
Increases element size	No	Yes
Used for	Element spacing	Content spacing

Que: 2

How margins create space outside the element and padding creates space inside.

Ans:

- **Margin:**
  - Margin creates space around the element, separating it from others.
  - **Example (Outside Space)**

```
<p style="margin: 20px; border: 2px solid black;">
   This paragraph has margin.
</p>
```

- **Padding:**
  - Padding creates space between the content and the border.
  - **Example:**

```
<p style="padding: 20px; border: 2px solid black;">
   This paragraph has padding.
</p>
```

- **Visual Representation**

```
+--------------------------+
|          Margin          |
|   +------------------+   |
|   |      Padding     |   |
|   |  Content Text    |   |
|   +------------------+   |
+--------------------------+
```

- **Key Points**
  - **Margin →** Space outside the element
  - **Padding →** Space inside the element
  - Margin separates elements
  - Padding improves readability of content
  - Both are essential for proper layout and design in CSS.

CSS: Pseudo-Class

Que: 1

Introduction to CSS Pseudo-Classes

Ans:

❖ **Introduction:**

- CSS pseudo-classes are used to define a special state of an HTML element.
- They allow you to apply styles when an element is hovered, clicked, focused, or visited, without using JavaScript.
- Pseudo-classes are written using a colon (:) followed by the state name.

- **Syntax:**

```
selector:pseudo-class {
    property: value;
}
```

➢ **Common CSS Pseudo-Classes**

1. **:hover**
   - Applies styles when the mouse pointer is placed over an element.

- **Example:**

```
a:hover {
    color: red;
}
```

   - Used for buttons, links, menus, etc.

2. **:focus**
   - Applies styles when an element (like input box) is selected or active for typing.

- **Example:**

```
input:focus {
    border: 2px solid blue;
}
```

- Improves user experience in forms.

3. **:active**
   - Applies styles when an element is clicked.

- **Example:**

```
button:active {
  background-color: green;
}
```

   - Visible while the mouse button is pressed.

4. **:visited**
   - Applies styles to visited links.

- **Example:**

```
a:visited {
  color: purple;
}
```

5. **:link**
   - Applies styles to unvisited links.

- **Example:**

```
a:link {
  color: blue;
}
```

Que: 2

Use of Pseudo-Classes Based on Element State

Ans:

❖ **Pseudo-classes allow styling elements based on their current state:**

Pseudo-Class	**Used When**
:hover	Mouse is over the element
:focus	Element is selected
:active	Element is clicked
:visited	Link already visited
:link	Link not visited

➢ **Example Using Multiple Pseudo-Classes**

```
a {
   text-decoration: none;
}

a:hover {
   color: red;
}

a:active {
   color: green;
}
```

- **Conclusion**

  CSS pseudo-classes help create interactive and user-friendly web pages by applying styles based on user actions without extra scripting.

CSS: ID and Class Selectors

Que: 1

Difference between id and class in CSS.

Ans:

❖ **Difference Between ID and Class in CSS**

- In CSS, id and class are used to select and style HTML elements.
- Both help apply CSS styles, but they are used in different situations.

1. **ID Selector**
   - An ID is used to identify one unique element on a webpage.
   - Each ID name must be unique (used only once).
   - Written using the **#** symbol in CSS.
   - **Example**

```
<h1 id="mainTitle">Welcome</h1>


#mainTitle {
   color: blue;
   text-align: center;
}
```

2. **Class Selector**
   - A class is used to style multiple elements.
   - Same class name can be used many times.
   - Written using the **.** symbol in CSS.

- **Example**

```
<p class="textStyle">Paragraph One</p>
<p class="textStyle">Paragraph Two</p>
.textStyle {
   color: green;
```

```
    font-size: 16px;
}
```

- **Comparison Table: ID vs Class**

Feature	ID	Class
Symbol in CSS	#	.
Usage	One element only	Multiple elements
Uniqueness	Must be unique	Can be reused
Priority	Higher priority	Lower priority
Common use	Layout, main sections	Reusable styles

- **Key Points**
  - Use ID for unique elements like header, footer, main section
  - Use Class for common styling like buttons, text, cards
  - One element can have multiple classes, but only one ID

- **Conclusion**
  - ID is best for unique styling
  - Class is best for reusable styling
  - Using them correctly makes CSS clean and efficient.

Que: 2

Usage scenarios for id (unique) and class (reusable).

Ans:

❖ **Usage Scenarios for ID and Class in CSS**

In CSS, id and class are used to apply styles to HTML elements, but they are chosen based on how many times the style is needed and the role of the element.

➢ **When to Use id (Unique)**

- Use an id when you need to style or target one specific, unique element on a webpage.

- **Common Usage Scenarios for id**
  - Main header of the page
  - Navigation bar
  - Footer section
  - Unique form or section
  - JavaScript targeting (single element)

- **Example**

```
<header id="mainHeader">My Website</header>


#mainHeader {
  background-color: lightblue;
  text-align: center;
}
```

  - Here, id is used because the header appears only once.

➢ **When to Use class (Reusable)**

- Use a class when the same style needs to be applied to multiple elements.

- **Common Usage Scenarios for class**
  - Buttons
  - Cards

- Paragraph styles
- Menu items
- Repeated form fields

- **Example**

```
<button class="btn">Save</button>

<button class="btn">Cancel</button>

.btn {

   background-color: green;

   color: white;

}
```

- Here, the class btn is reused for multiple buttons.

➢ **Real-World Example (ID + Class Together)**

```
<div id="content">

  <p class="textStyle">Paragraph 1</p>

  <p class="textStyle">Paragraph 2</p>

</div>
```

- id="content" → Unique section
- class="textStyle" → Reusable paragraph style

➢ **Comparison**

Feature	**ID**	**Class**
Use case	Unique element	Multiple elements
Reusability	No	Yes
CSS symbol	#	.
Best for	Layout sections	Repeated styles

- **Conclusion**
  - Use id for unique page elements
  - Use class for reusable styles
  - This approach keeps your CSS clean, organized, and scalable.

Introduction to Client-Server Architecture

Que: 1

Overview of client-server architecture.

Ans:

❖ **Overview of Client–Server Architecture**

Client–Server Architecture is a model where clients request services and a server provides those services over a network.

1. **Client**
   - The client is a device or application that sends a request to the server.

➢ **Examples:**

- Web browser (Chrome)
- Mobile app
- Desktop application

➢ **Example request:**

- User opens a website and sends a request for a web page.

2. **Server**
   - The server is a computer or program that receives the client request, processes it, and sends back a response.

➢ **Example:**

- A web server like Apache Tomcat processes requests from the browser.

3. **Working Process**
   - Client sends a request to the server.
   - Server processes the request.
   - Server sends a response back to the client.

➢ **Example in Servlet:**

- Browser sends request to servlet
- Servlet processes data
- Server returns HTML response

4. **Example**
   - Client → Web Browser

- Server → Web Server

➢ User opens website → Browser sends request → Server processes → Web page displayed.

Que: 2

Difference between client-side and server-side processing.

Ans:

❖ **Difference Between Client-Side and Server-Side Processing**

Feature	**Client-Side Processing**	**Server-Side Processing**
Definition	Processing done on the user's device (browser)	Processing done on the web server
Where it runs	Client computer / browser	Server machine
Purpose	User interface and quick validations	Business logic and database operations
Security	Less secure	More secure
Speed	Faster (no server request needed)	Slower (request sent to server)
Example	Form validation using JavaScript	Login authentication using Servlet

➢ **Example**

- **Client-Side Example**

- Checking if email field is empty using JavaScript before submitting the form.

- **Server-Side Example**

- Verifying login data with database using JDBC on a server like Apache Tomcat.

Que: 3

Roles of Client, Server, and Communication Protocols.

Ans:

1. **Client**

   A client is a device or application that sends a request to the server and waits for a response.

- **Examples:**
  - Web browser
  - Mobile application
- **Example:**
  - A browser requests a webpage from a server.

2. **Server**

   A server is a computer or program that receives the client request, processes it, and sends back a response.

- **Example:**
  - A server running Servlet processes user requests and returns data using a server like Apache Tomcat.

3. **Communication Protocols**

   Communication protocols are rules that define how data is transmitted between client and server.

- **Common protocol:**
  - HTTP
- **Example flow:**
  - Client sends HTTP request
  - Server processes request
  - Server sends HTTP response
- **Short Interview Answer**

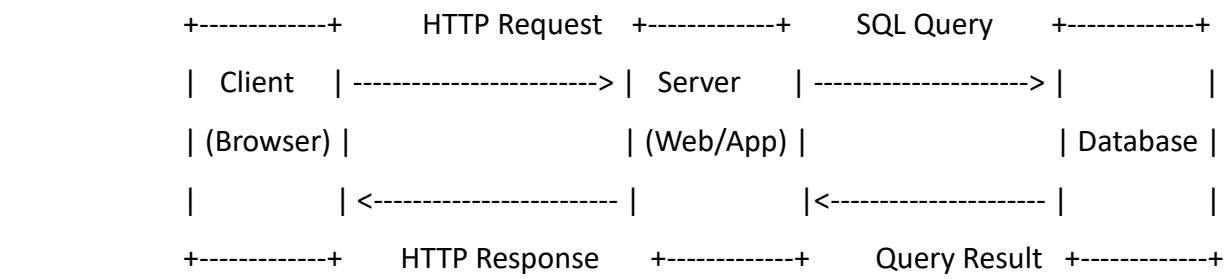
  The client sends requests, the server processes the requests and returns responses, and communication protocols like HTTP define the rules for data exchange between them.

Lab Exercise: 1

Create a diagram explaining client-server communication flow and explain how a request is processed by the server and sent back to the client.

Ans:

❖ **Client–Server Communication Diagram**



➢ **Explanation**

1. **Client Request**

   The user uses a browser (client) to send an HTTP request to the server (for example clicking a submit button on a form).

2. **Server Processing**

   The web server or application server (like Tomcat) receives the request and processes it using backend code such as Servlets or APIs.

3. **Database Interaction**

   If data is required, the server sends an SQL query to the database and retrieves the result.

4. **Response to Client**

   The server prepares the response (HTML/JSON) and sends an HTTP response back to the client, which the browser displays to the user.

➢ **In short:**

Client sends request → Server processes it → Database may be used → Server sends response back to client.

HTTP Protocol Overview with Request and Response Headers

Que: 1

Introduction to the HTTP protocol and its role in web communication.

Ans:

❖ **Introduction to the HTTP Protocol**

- HTTP is a communication protocol used to transfer data between a client (browser) and a web server on the internet.
- It is the foundation of web communication and is used to load web pages.

➢ **Role of HTTP in Web Communication**

1. **Request–Response Model**

   HTTP works on a request–response system.

   - Client sends a request
   - Server processes it
   - Server sends a response

2. **Data Transfer**

   HTTP transfers different types of data such as:

   - HTML pages
   - Images
   - Videos
   - JSON data

3. **Communication Between Browser and Server**

   HTTP allows the browser to communicate with server applications like Servlet running on servers such as Apache Tomcat.

➢ **Example**

User types a website URL in browser →

Browser sends HTTP request →

Server processes request →

Server sends HTTP response with webpage.

Que: 2

Explanation of HTTP request and response headers.

Ans:

❖ **HTTP Request and Response Headers**

In HTTP, headers are extra information sent between the client (browser) and the server during communication.

Headers help the server and client understand how to process the request and response.

1. **HTTP Request Headers**
   - Request headers are sent by the client (browser) to the server.
   - They contain information about the request and the client.

   ➢ **Examples**
   - Host → Domain name of the website
   - User-Agent → Information about the browser
   - Accept → Type of data the client can receive
   - Cookie → Stored user information

   ➢ **Example request header:**

   GET /index.html HTTP/1.1

   Host: www.example.com

   User-Agent: Chrome

   Accept: text/html

2. **HTTP Response Headers**
   - Response headers are sent by the server to the client after processing the request.
   - They contain information about the server response.

   ➢ **Examples**
   - Content-Type → Type of data returned (HTML, JSON, etc.)
   - Content-Length → Size of the response
   - Set-Cookie → Store cookie in browser
   - Server → Information about the server

- **Example response header:**

  HTTP/1.1 200 OK

  Content-Type: text/html

  Content-Length: 305

  Server: Apache

- **Role in Web Applications**

  In Java web applications like Servlet running on Apache Tomcat, headers are used to:

  - Send user data
  - Manage sessions
  - Control caching
  - Define response type

J2EE Architecture Overview

Que: 1

Introduction to J2EE and its multi-tier architecture.

Ans:

❖ **Introduction to J2EE**

- Java 2 Platform Enterprise Edition (J2EE) is a platform used to build large-scale enterprise web applications using Java.
- It provides technologies like:
  - Servlet
  - JavaServer Pages
  - JDBC
- These technologies help developers create secure, scalable, and distributed web applications.

➢ **J2EE Multi-Tier Architecture**

J2EE follows a multi-tier architecture, which means the application is divided into multiple layers. Each layer has a specific role.

1. **Client Tier**

   This is the user interface where the user interacts with the application.

   ➢ **Examples:**
   - Web browser
   - Mobile app

   ➢ **Technologies:**
   - HTML
   - CSS
   - JavaScript

2. **Web Tier**

   This layer handles client requests and responses.

   ➢ **Technologies:**
   - Servlet
   - JavaServer Pages

   ➢ **Example:**
   - Login form request goes to servlet.

3. **Business Tier**

   This layer contains the business logic of the application.

   - **Example:**
     - User authentication
     - Order processing
     - Payment logic

4. **Data Tier**

   This layer stores and manages application data.

   - **Technologies:**
     - Databases like MySQL
     - Java connects to the database using:
     - JDBC

- **Example Flow:**

  User opens website → request goes to servlet → business logic processes data → database returns result → response sent to browser.

Que: 2

Role of web containers, application servers, and database servers.

Ans:

❖ **Role of Web Containers, Application Servers, and Database Servers**

1. **Web Container**

   A web container manages web components like Servlet and JavaServer Pages.

   - **Responsibilities:**
     - Handles HTTP requests and responses
     - Manages servlet life cycle
     - Provides security and session management
   - **Example:**
     - Apache Tomcat

2. **Application Server**

   An application server provides a complete environment to run enterprise applications.

   - **Responsibilities:**
     - Runs business logic
     - Manages transactions
     - Handles security and scalability
     - Supports enterprise technologies
   - **Example:**
     - WildFly

3. **Database Server**

   A database server stores and manages application data.

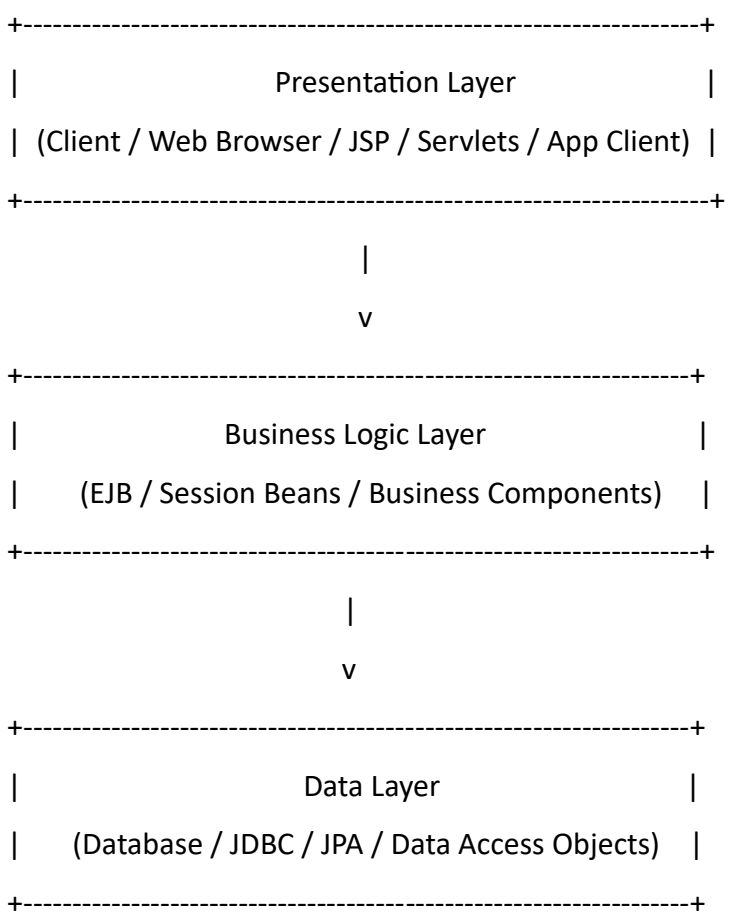
   - **Responsibilities:**
     - Store data
     - Retrieve data
     - Manage database queries
   - **Example:**
     - MySQL
     - Java applications connect to the database using JDBC.

Lab Exercise: 1

Draw and explain the J2EE architecture, labeling the layers like the presentation layer, business logic layer, and data layer.

Ans:

❖ **J2EE Architecture Diagram**



➢ **Layer-wise Explanation**

1. **Presentation Layer**

- **Purpose:** Handles user interaction and displays information.
- **Components:**
  - **Web Clients**: Browser (HTML, JavaScript)
  - **Servlets / JSP:** Dynamic web content
  - **App Clients**: Standalone Java applications
- **Responsibility:** Receives input from users, forwards requests to business layer, and displays results.

2. **Business Logic Layer**

- **Purpose:** Contains the core application logic.
- **Components:**
  - EJBs (Enterprise JavaBeans)
  - Session Beans (Stateless / Stateful)
  - Other Business Components
- **Responsibility:**
  - Processes client requests
  - Implements business rules
  - Interacts with data layer to fetch or store data

3. **Data Layer**

- **Purpose:** Manages data storage and retrieval.
- **Components:**
  - Relational Databases (MySQL, Oracle)
  - JDBC / JPA / Hibernate
  - DAO (Data Access Objects)
- **Responsibility:**
  - Perform SQL operations
  - Return data to business layer
  - Ensure persistence and data integrity

➢ **Summary (Flow)**

- **Client** → Presentation Layer: Sends request (e.g., form submission).
- **Presentation Layer** → Business Layer: Forwards request to process business logic.
- **Business Layer** → Data Layer: Queries or updates database.
- **Data Layer** → Business Layer: Sends data/results.
- **Business Layer** → Presentation Layer: Returns processed results.
- **Presentation Layer** → Client: Displays response to the user.

Web Component Development in Java (CGI Programming)

Que: 1

Introduction to CGI (Common Gateway Interface).

Ans:

❖ **Introduction to CGI**

- Common Gateway Interface (CGI) is a standard method used to connect web servers with external programs to generate dynamic web content.
- Before technologies like Servlet, CGI was commonly used to process user requests on websites.

➢ **How CGI Works**

1. A user sends a request from the browser.
2. The web server runs a CGI program.
3. The program processes the request.
4. The server sends the result back to the browser.

➢ **Example**

User fills a form on a website →

Server runs a CGI program →

Program processes data →

Response is sent back to the browser.

Languages Used in CGI

➢ **CGI programs can be written in languages like:**

- C
- C++
- Perl
- Python

➢ **Limitation of CGI**

- Each request creates a new process, which makes it slow and less efficient.
- That is why modern web applications use technologies like Servlet.

Que: 2

Process, advantages, and disadvantages of CGI programming.

Ans:

❖ **CGI Programming**

Common Gateway Interface is a technique used to connect a web server with external programs to generate dynamic web content.

1. **Process of CGI Programming**
    1. User sends a request from the browser.
    2. The web server receives the request.
    3. The server executes a CGI program.
    4. The CGI program processes the request (form data, etc.).
    5. The program sends output (HTML page) to the server.
    6. The server sends the response back to the client.
2. **Advantages of CGI**
    - Language Independent → Can be written in C, C++, Python, Perl, etc.
    - Simple to implement
    - Works with many web servers
    - Can create dynamic web pages
3. **Disadvantages of CGI**
    - **Slow performance** because a new process is created for every request
    - **High server load** when many users access the website
    - **Not scalable** for large web applications
    - **More memory** consumption

→ Because of these problems, modern technologies like Servlet are used instead of CGI.

Servlet Programming: Introduction, Advantages, and Disadvantages

Que: 1

Introduction to servlets and how they work.

Ans:

❖ **Introduction to Servlets**

- Servlet is a Java program that runs on a web server and handles requests from a client (browser) and sends a response back.
- Servlets are used to create dynamic web applications such as login systems, registration forms, and product management systems.
- Servlets run on a web server like Apache Tomcat.

➢ **How Servlets Work**

1. User sends a request from the browser.
2. The request goes to the web server (Tomcat).
3. The server forwards the request to the servlet.
4. The servlet processes the request (business logic, database operations).
5. The servlet sends a response back to the server.
6. The server sends the response to the browser.

➢ **Example Flow**

Browser → Request → Server → Servlet → Process Data → Response → Browser(client)

Que: 2

Advantages and disadvantages compared to other web technologies.

Ans:

- **Advantages of Servlet**
  1. **Better Performance**

     Servlets are faster than Common Gateway Interface because they use threads instead of creating a new process for every request.

  2. **Platform Independent**

     Since they are written in Java, they can run on any system that supports Java.

  3. **More Secure**

     Servlets run inside a web container like Apache Tomcat, which provides security features.

  4. **Reusable and Scalable**

     Servlets support modular and scalable web application development.

  5. **Easy Database Connectivity**

     Servlets can connect to databases using JDBC.

- **Disadvantages of Servlets**
  1. **Hard to Design UI**

     Servlets generate HTML using Java code, which makes UI design difficult.

  2. **More Code**

     Compared to technologies like JavaServer Pages, servlets require more coding.

  3. **Requires Web Server**

     Servlets need a server like Apache Tomcat to run.

- **Short Interview Answer:** Servlets are faster and more secure than CGI because they use threads instead of creating new processes. However, they require more coding and are not ideal for designing user interfaces.

Servlet Versions, Types of Servlets

Que: 1

History of servlet versions.

Ans:

❖ **History of Servlet Versions**

Servlet was developed to create dynamic web applications using Java.

➢ **Important Servlet Versions**

1. **Servlet 1.0 (1997)**
   - First version introduced
   - Basic support for web applications
2. **Servlet 2.0 (1998)**
   - Added session management
   - Improved performance
3. **Servlet 2.3 (2001)**
   - Introduced filters and event listeners
4. **Servlet 2.5 (2005)**
   - Simplified web application development
5. **Servlet 3.0 (2009)**
   - Introduced annotations like @WebServlet
   - Reduced use of web.xml
6. **Servlet 4.0 (2017)**
   - Added HTTP/2 support
   - Improved web communication

➢ Servlets usually run on servers like Apache Tomcat.

Que: 2

Types of servlets: Generic and HTTP servlets.

Ans:

❖ **Types of Servlets**

In Servlet, there are mainly two types of servlets.

1. **Generic Servlet**

GenericServlet is a protocol-independent servlet class.

➢ **Features:**

- Works with any protocol (not only HTTP)
- Belongs to javax.servlet package
- Requires overriding the service() method

➢ **Example use:**

- When the servlet needs to work with protocols other than HTTP.

2. **HTTP Servlet**

HttpServlet is a servlet designed specifically for HTTP protocol.

➢ **Features:**

- Used for web applications
- Belongs to javax.servlet.http package

➢ **Provides methods like:**

- doGet()
- doPost()
- doPut()
- doDelete()

➢ **Example use:**

- Login forms
- Registration forms

➢ Web applications running on servers like Apache Tomcat.

Difference between HTTP Servlet and Generic Servlet

Que: 1

Detailed comparison between HttpServlet and GenericServlet.

Ans:

❖ **Detailed Comparison Between GenericServlet and HttpServlet**

Feature	**GenericServlet**	**HttpServlet**
Definition	A protocol-independent servlet	A servlet designed specifically for HTTP protocol
Package	javax.servlet	javax.servlet.http
Protocol Support	Supports any protocol	Supports only HTTP
Main Method	Uses service() method	Uses doGet(), doPost(), doPut(), doDelete()
Usage	Rarely used in web applications	Mostly used for web applications
Flexibility	Can work with different protocols	Works only with HTTP requests
Complexity	Simpler but less useful for web apps	More useful for web-based applications

➢ **Example**

- **GenericServlet Method**

  public void service(ServletRequest req, ServletResponse res)

- **HttpServlet Methods**

  protected void doGet(HttpServletRequest req, HttpServletResponse res)

  protected void doPost(HttpServletRequest req, HttpServletResponse res)

➢ **Usage**

Most modern web applications use HttpServlet and run on servers like Apache Tomcat.

Servlet Life Cycle

Que: 1

Explanation of the servlet life cycle: init(), service(), and destroy() methods.

Ans:

❖ **Servlet Life Cycle**

The Servlet Life Cycle describes how a Servlet is created, used, and destroyed inside a web server like Apache Tomcat.

There are 3 main methods in the servlet life cycle.

1. **init() Method**
   - Called only once when the servlet is first loaded into memory.
   - Used for initialization tasks like creating database connections.
   - **Example:**

```
public void init()
{
  // initialization code
}
```

2. **service() Method**
   - Called every time a client sends a request.
   - Processes the request and sends a response.
   - **Example:**

```
public void service(ServletRequest req, ServletResponse res)
{
  // request processing
}
```

3. **destroy() Method**
   - Called once when the servlet is removed from memory or the server is stopped.
   - Used to release resources like closing database connections.

- **Example:**

```
public void destroy()
{
  // cleanup code
}
```

➢ **Flow of Servlet Life Cycle**

- Server loads servlet → init()
- Client request comes → service()
- Server stops servlet → destroy()

➢ **Short Interview Answer**

The servlet life cycle consists of three methods: init() for initialization, service() for handling client requests, and destroy() for releasing resources when the servlet is removed.

Creating Servlets and Servlet Entry in web.xml

Que: 1

How to create servlets and configure them using web.xml.

Ans:

❖ **How to Create Servlets and Configure Them Using web.xml**

A Servlet is created using Java and configured in the web.xml file so the server knows how to access it.

Servlets usually run on a server like Apache Tomcat.

1. **Steps to Create a Servlet**
   - **Step 1: Create a Java Class**

     Create a class that extends HttpServlet.
   - **Example:**

```
import java.io.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;


public class HelloServlet extends HttpServlet

{

  protected void doGet(HttpServletRequest req, HttpServletResponse res)

  throws ServletException, IOException

  {

    res.setContentType("text/html");

    PrintWriter out = res.getWriter();

    out.println("<h1>Hello Servlet</h1>");

  }

}
```

2. **Configure Servlet in web.xml**

   web.xml is the deployment descriptor file used to map a servlet with a URL.

   - **Example configuration:**

     <web-app>

     <servlet>

     <servlet-name>HelloServlet</servlet-name>

     <servlet-class>HelloServlet</servlet-class>

     </servlet>

     <servlet-mapping>

     <servlet-name>HelloServlet</servlet-name>

     <url-pattern>/hello</url-pattern>

     </servlet-mapping>

     </web-app>

3. **How It Works**

   1. User enters **URL**

      http://localhost:8080/project/hello
   2. Server checks **web.xml**
   3. URL /hello is mapped to **HelloServlet**
   4. Servlet runs and sends response to browser.

➢ **Short Interview Answer**

A servlet is created by extending HttpServlet and writing methods like doGet() or doPost(). It is configured in the web.xml file by defining the servlet and mapping it to a URL pattern.

Logical URL and ServletConfig Interface

Que: 1

Explanation of logical URLs and their use in servlets.

Ans:

❖ **Logical URLs in Servlets**

In Servlet, a logical URL is a URL pattern used to access a servlet instead of using the actual class name.

It creates a mapping between a URL and a servlet class.

➢ **Why Logical URLs Are Used**

1. **Easy access**

- Users can access servlets with a simple URL.

2. **Security**

- The real servlet class name is hidden.

3. **Flexibility**

- If the class name changes, the URL can remain the same.

➢ **Example Using web.xml**

<servlet>

<servlet-name>LoginServlet</servlet-name>

<servlet-class>LoginServlet</servlet-class>

</servlet>

<servlet-mapping>

<servlet-name>LoginServlet</servlet-name>

<url-pattern>/login</url-pattern>

</servlet-mapping>

Here:

- LoginServlet → Actual servlet class
- /login → Logical URL
- **User accesses the servlet like this:**

  http://localhost:8080/project/login

➢ **Working**

Browser request → Server checks URL pattern →

Server calls the mapped HttpServlet →

Servlet processes request → Response sent back.

➢ Usually servlets run on servers like Apache Tomcat.

➢ **Short Interview Answer**

A logical URL is a URL pattern used to access a servlet without exposing its actual class name. It maps a user request to the servlet using web.xml or annotations.

Que: 2

Overview of ServletConfig and its methods.

Ans:

❖ **What is ServletConfig?**

- ServletConfig is an interface in the Servlet API that provides configuration information for a servlet.
- It allows a servlet to:
  - Get initialization parameters from web.xml
  - Access the ServletContext
  - Know the servlet name
- ServletConfig object is created by the servlet container (like Apache Tomcat) and passed to the servlet during initialization.
- It is available inside the init() method.

❖ **Why ServletConfig is Used**

- **ServletConfig is used to:**
  1. Read init parameters
  2. Configure servlet behavior
  3. Share configuration data
  4. Access ServletContext
- **Example use cases:**
  - Database URL
  - Username/password
  - Application configuration

❖ **ServletConfig Methods**

1. getInitParameter(String name)

- Returns the value of initialization parameter.

- **Syntax**

  String value = config.getInitParameter("parameterName");

- **Example in web.xml**

  <servlet>

```
    <servlet-name>MyServlet</servlet-name>
    <servlet-class>com.demo.MyServlet</servlet-class>

    <init-param>
      <param-name>username</param-name>
      <param-value>admin</param-value>
    </init-param>
  </servlet>
```

- **Servlet Code**

```
public void init(ServletConfig config) throws ServletException {
  String user = config.getInitParameter("username");
}
```

2. getInitParameterNames()

- Returns all initialization parameter names.

- **Return Type:**

```
Enumeration<String>
```

- **Example**

```
Enumeration<String> params = config.getInitParameterNames();
while(params.hasMoreElements()){
  String name = params.nextElement();
}
```

3. getServletContext()

- Returns the ServletContext object.
- ServletContext represents the entire web application environment.

- **Example**

```
ServletContext context = config.getServletContext();
```

- **Uses:**
  - Share data between servlets
  - Read global parameters
  - Access server resources

4. getServletName()

- Returns the name of the servlet defined in web.xml.

- **Example**

  String name = config.getServletName();

- **If in web.xml:**

  <servlet-name>LoginServlet</servlet-name>

- **Output:**

  LoginServlet

➢ **Complete Example**

```
import jakarta.servlet.*;
import java.io.*;
public class MyServlet extends GenericServlet {

   public void service(ServletRequest req, ServletResponse res)
   throws ServletException, IOException {
      ServletConfig config = getServletConfig();

      String user = config.getInitParameter("username");

      PrintWriter out = res.getWriter();
      out.println("Username: " + user);
   }
```

}

- **ServletConfig vs ServletContext**

Feature	**ServletConfig**	**ServletContext**
Scope	Single servlet	Entire application
Parameters	Servlet specific	Global parameters
Access	Only that servlet	All servlets
Created by	Container	Container

- **Key Interview Points**
  - ServletConfig is servlet-specific configuration
  - Created by Servlet Container
  - Used in init() method
  - Reads init parameters from web.xml
  - Helps access ServletContext

RequestDispatcher Interface: Forward and Include Methods

Que: 1

Explanation of RequestDispatcher and the forward() and include() methods.

Ans:

❖ **RequestDispatcher**

- RequestDispatcher is an interface used in Servlets to send a request from one resource to another resource (Servlet, JSP, or HTML) inside the server.
- It is part of the Jakarta Servlet API.

➢ **Main Methods**

1. **forward()**

- Transfers the request completely to another resource.
- The current servlet stops execution.
- Browser URL does not change.

- **Syntax**

```
RequestDispatcher rd = request.getRequestDispatcher("page.jsp");
rd.forward(request, response);
```

- **Example:** LoginServlet → Dashboard.jsp

2. **include()**

- Includes another resource's output in the current response.
- The original servlet continues execution.

➢ **Syntax**

```
RequestDispatcher rd = request.getRequestDispatcher("header.jsp");
rd.include(request, response);
```

➢ **Example:** Header.jsp + Content + Footer.jsp

- **Difference between forward() and include() method**

forward()	**include()**
Transfers control completely	Adds content
Servlet execution stops	Servlet continues
Used for page redirection inside server	Used for layout (header/footer)

- **Interview Line:**

  RequestDispatcher is used for server-side communication between servlets or JSP pages using forward() and include() methods.

ServletContext Interface and Web Application Listener

Que: 1

Introduction to ServletContext and its scope.

Ans:

❖ **ServletContext**

- ServletContext is an interface that provides information about the entire web application.
- It is created once per web application and is shared by all servlets.

➢ **Scope of ServletContext**

- Its scope is application-wide.
- Data stored in ServletContext can be accessed by all servlets in the application.
- **Example:**

  ServletContext context = getServletContext();

  context.setAttribute("company","ABC Tech");

- **Another servlet can read it:**

  String name = (String) context.getAttribute("company");

➢ **Common Methods**

- getInitParameter() → get initialization parameter
- setAttribute() → store data
- getAttribute() → retrieve stored data
- removeAttribute() → remove data

➢ **Interview Answer:**

ServletContext is an interface that represents the web application environment and allows data sharing between multiple servlets across the entire application.

Que: 2

How to use web application listeners for lifecycle events.

Ans:

❖ **Web Application Listeners**

- A Servlet Listener is used to listen for events in a web application, such as application start, session creation, or request events.
- Listeners are part of the Jakarta Servlet API.

➢ **Purpose**

- Listeners are used to perform actions when lifecycle events occur in a web application.
- **Example events:**
  - Application start
  - Session start/end
  - Request start/end

➢ **Common Listener Types**

1. ServletContextListener

- Executes when application starts or stops.
- **Important methods:**
  - contextInitialized() → when application starts
  - contextDestroyed() → when application stops
- **Example:**

```
public class AppListener implements ServletContextListener {
   public void contextInitialized(ServletContextEvent e){
      System.out.println("Application Started");
   }

   public void contextDestroyed(ServletContextEvent e){
      System.out.println("Application Stopped");
   }
```

}

2. **HttpSessionListener**
   - Executes when session is created or destroyed.
   - **Methods:**
     - sessionCreated()
     - sessionDestroyed()

➢ **How to Configure Listener**

1. **Using Annotation**

   @WebListener

   public class AppListener implements ServletContextListener

2. **Using web.xml**

   <listener>

   <listener-class>AppListener</listener-class>

   </listener>

➢ **Interview Answer:**

Web application listeners are used to handle lifecycle events such as application start/stop or session creation/destruction in a servlet-based web application.

Java Filters: Introduction and Filter Life Cycle

Que: 1

What are filters in Java and when are they needed?

Ans:

❖ **Filters in Java (Servlet)**

- A Servlet Filter is used to intercept and process client requests and server responses before they reach the servlet or after they leave the servlet.
- Filters are part of the Jakarta Servlet technology.

➢ **Why Filters are Needed**

Filters are used for common tasks before or after request processing, such as:

- Authentication (check login)
- Logging requests
- Data validation
- Compression
- Encryption
- Modifying request or response

- **Example:**

  Before opening a dashboard page, a filter checks whether the user is logged in or not.

➢ **Filter Life Cycle Methods**

1. **init()** → Called once when filter is created
2. **doFilter()** → Processes request and response
3. **destroy()** → Called when filter is removed

- **Example:**

```
public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain){
   System.out.println("Request received");
   chain.doFilter(req,res);
}
```

- **Interview Answer:**

  A filter in Java is used to intercept and process requests and responses before they reach the servlet, mainly for tasks like authentication, logging, and validation.

Que: 2

Filter lifecycle and how to configure them in web.xml.

Ans:

❖ **Filter Life Cycle**

- In Servlet Filter, the lifecycle contains three main methods.

1. **init()**

- Called once when the filter is created.
- Used for initialization.

```
public void init(FilterConfig config)
```

2. **doFilter()**

- Called every time a request comes.
- Used to process request and response before sending to servlet.

```
public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
```

3. **destroy()**

- Called once when the filter is removed.
- Used to release resources.

```
public void destroy()
```

➢ **Configure Filter in web.xml**

- Filters are configured in web.xml.
- **Example:**

```
<filter>
  <filter-name>AuthFilter</filter-name>
  <filter-class>com.demo.AuthFilter</filter-class>
</filter>

<filter-mapping>
  <filter-name>AuthFilter</filter-name>
```

<url-pattern>/dashboard</url-pattern>

</filter-mapping>

- **Explanation:**
  - **<filter>** → defines the filter class
  - **<filter-mapping>** → specifies which URL the filter applies to

➢ **Interview Answer:**

Filter lifecycle contains three methods: init(), doFilter(), and destroy(). Filters are configured in web.xml using <filter> and <filter-mapping> tags.

JSP Basics: JSTL, Custom Tags, Scriplets, and Implicit Objects

Que: 1

Introduction to JSP and its key components: JSTL, custom tags,scriplets, and implicit objects.

Ans:

❖ **Introduction to JSP**

- JavaServer Pages is a server-side technology used to create dynamic web pages using Java code inside HTML.
- JSP runs on servers like Apache Tomcat and internally converts JSP into a Servlet.

- **Example:**

  <%= "Hello User" %>

➢ **Key Components of JSP**

1. **JSTL**

- JSTL provides standard tags to perform common tasks like loops and conditions.
- **Example:**

  <c:forEach var="i" begin="1" end="5">

  ${i}

  </c:forEach>

- **Used for:**
  - loops
  - conditions
  - formatting

2. **Custom Tags**

- Custom tags are user-defined tags used to reuse code in JSP.
- **Example:**

  <mytag:header />

- **Purpose:**
  - reduce Java code in JSP
  - reusable components

3. **Scriptlets**

- Scriptlets allow Java code inside JSP.
- **Example:**

```
<%
int a = 10;
out.println(a);
%>
```

- **Note:** Scriptlets are not recommended in modern JSP.

4. **Implicit Objects**

- JSP provides predefined objects automatically.
- **Common implicit objects:**
  - request
  - response
  - session
  - application
  - out
  - pageContext
- **Example:**

```
<%= request.getParameter("name") %>
```

➢ **Interview Answer:**

JSP is a server-side technology used to create dynamic web pages using Java. Its key components include JSTL, custom tags, scriptlets, and implicit objects.

Session Management and Cookies

Que: 1

Overview of session management techniques: cookies, hidden form fields, URL rewriting, and sessions.

Ans:

❖ **Session Management Techniques**

In web applications using Jakarta Servlet, session management is used to maintain user information between multiple requests because HTTP is stateless.

➢ **Main session management techniques:**

1. **Cookies**

- A HTTP Cookie is small data stored in the user's browser.
- **Example:**

```
Cookie c = new Cookie("user","ram");

response.addCookie(c);
```

- **Used for:**
  - login information
  - user preferences

2. **Hidden Form Fields**

- Data is stored in hidden input fields in HTML forms.
- **Example:**

```
<input type="hidden" name="userid" value="101">
```

- **Used when:**
  - data must be passed between form submissions.

3. **URL Rewriting**

- Session data is added directly to the URL.
- **Example:**

```
example.com/home?userid=101
```

- **Used when:**

- cookies are disabled in the browser.

4. Sessions

- A HttpSession stores user data on the server side.
- **Example:**

  HttpSession session = request.getSession();

  session.setAttribute("user","ram");

- **Used for:**
  - login systems
  - shopping carts

➢ **Interview Answer:**

Session management techniques include Cookies, Hidden Form Fields, URL Rewriting, and HttpSession, which help maintain user data across multiple HTTP requests.

Que: 2

How to track user sessions in web applications.

Ans:

❖ **Main Ways to Track User Sessions**

1. **Cookies**

- Using HTTP Cookie to store user information in the browser.
- **Example:**

  Cookie c = new Cookie("username","ram");

  response.addCookie(c);

2. **URL Rewriting**

- Session ID is added in the URL.
- **Example:**

  example.com/home;jsessionid=12345

3. **Hidden Form Fields**

- User data is passed using hidden fields in forms.
- **Example:**

  <input type="hidden" name="userid" value="101">

4. **HttpSession (Most Common)**

- Using HttpSession to store session data on the server.
- **Example:**

  HttpSession session = request.getSession();

  session.setAttribute("user","ram");

➢ **Interview Answer:**

User sessions in web applications are tracked using Cookies, URL Rewriting, Hidden Form Fields, and HttpSession. HttpSession is the most commonly used method because it stores user data on the server.